# Phase 8 | Mock Interview

*A practice conversation for presenting Ready Scout accurately and confidently.*



**Interviewer: Give me the overview.**

Candidate: Ready Scout is a disaster-preparedness learning platform. The main web app uses React with Vite, an Express API and MongoDB Atlas. It has learning pages, earthquake practice, alert views, a nearby-services locator and a chatbot. The chatbot is a separate Flask service that calls Gemini. I treat it as an educational prototype because the external data and credentials are not guaranteed emergency infrastructure.

**Interviewer: Why did you move to Vite?**

Candidate: The frontend is a client-side React application using React Router and a separate Express API. Vite gives a lightweight dev server and static production build, while Express owns backend routes. That removed the need for Next.js server and route features in this setup.

**Interviewer: Explain registration and login.**

Candidate: The form posts to Express. Registration validates fields, optionally uploads an image, creates a Mongoose user and returns a signed JWT. The model hashes passwords with bcryptjs. Login compares the supplied password with the hash. Protected routes verify a Bearer JWT and load the user. I would improve token storage and the refresh flow.

**Interviewer: What is the nearby-service architecture?**

Candidate: The browser obtains a location, with a default fallback if permission is unavailable. React calls Express. Express validates coordinates, coalesces duplicate requests and caches normalized Geoapify Places results. The frontend sorts by Haversine distance and renders the list and markers through Leaflet. Provider configuration and OSM coverage are limitations.

**Interviewer: Why separate the chatbot into Flask?**

Candidate: It isolates the Gemini integration in Python and keeps that API key on the service side. The trade-off is an extra deployment and failure boundary. I would add an explicit timeout, rate limiting, authentication, health checks and controlled error messages.

**Interviewer: How are alerts presented?**

Candidate: Express fetches SACHET summaries and normalizes them. The detail route retrieves CAP XML, parses its info fields and returns readable JSON. The UI polls summaries and loads details. Separate USGS and GDACS utilities exist in the frontend, and I would verify whether each feed is currently wired to a visible component before claiming it is active.

**Interviewer: The nearby feature fails in production. What do you check?**

Candidate: I trace the browser request and inspect its status and sanitized error. Then I verify the deployed backend revision, server-side key, key restrictions, quota, network timeout, request format and category list. I compare deployed environment settings with local without exposing the key. I also check whether a stale result can be served.

**Interviewer: What would you improve first?**

Candidate: I would improve security and observability: remove tokens from redirect URLs, redesign refresh token handling, validate configuration at startup, add upload limits and create integration tests for provider failures. Then I would add visible freshness and service health for alerts and nearby data.

## Follow-up practice

- What should happen if there are no active alerts versus the alert provider being down?
- How would you test OAuth callback behavior without using production credentials?
- What data belongs in the user collection, and what should never be logged?
- Which nearby data does the client cache, and how does the user know when it is stale?

- What would be required before positioning the locator as dependable emergency guidance?